

Client Ops Portal

OTUS C# ASP.NET Core



**Меня хорошо видно
& слышно?**



Защита проекта

Тема: CRM-система Client Ops Portal



Огурцова Александра

Место работы: Белтелеком.

План защиты

Цель и задачи проекта

Какие технологии использовались

Что получилось

Выводы

Вопросы и рекомендации

Цель и задачи проекта

Цель проекта: создание CRM-системы для ведения подписок на телекоммуникационные услуги.

Задачи проекта:

1. Спроектировать и реализовать микросервисную архитектуру
2. Организовать оптимальное хранение данных
3. Настроить асинхронную и синхронную связь между сервисами
4. Автоматизировать сборку, тесты и деплой (CI/CD)



Что планировалось

Для визуализации функционала CRM-системы был разработан прототип

Прототип: <https://raw.github.com/alex-makhina/client-ops-portal/CreatePrototype/Documentation/Prototype.html>

Первоначальная схема БД:

https://mermaid.live/edit#pako:eNrVVkuP2jAQ_iuWpb2xCLo8ltxYHqiqXaGye6m4mHgSrCZ2ZDvVUuC_13kSSKJlS9ttcwDbM9_M-POMxztsCwrYwiDHjLiS-CuOzPesQO6SYfS5laNoRtFiflxTWjLuosmLBsmJN6OJ6JD8Jb_DteDANXrFVLoWOUXTsosZNTaYw2yimeCPob82eiWtKZNKPxIfSpJpPebwmVHgQaVoaNsi5DpxdpRSokEzH9BlghnSoS4hU8nDtgL1HNAaVCrJUlbFZDASXEtiv0ph5jxVP487hWTnMZ3nR3QS3wO4jL_NrEI24bRG8re4WIL8zmy4klrKYx2DsiULojz677b_RCRznlVH-09nAGzmEw8tpOG3ZDvjvViab-fselzhOg_hwpshr4Lp_KLoUlGBsetif590L_D0kSkt5PZCuk4YLu4ceOibqy1af9oGcCZYaqJD9WYWDUzqf4-rpYbgF_jKmC7Rds5O1kNAKeJetfd8FxM_8MQW4Mp-aSJ1nPMO8Ee65EloXXHDBCYhfNNn3jEhbm7QF_DiB4PasECdvkf2-9tbsTs2Vwut8laoFT5tNKnaSTIFqozXaBZK5cxc7qrOpm0UCOM5oGCqDkLBYRyOURfIFZgssyNoFMu3SmSmVW8gLqW0ZGWwFBLOZVEXI5DggFRliwwaZ3ME2ov8nGKy1ZkGiIxyWsl0cAO7kiFsaRICA_sgFRJNcVxMZssbMBmMU-Jl6OnI7sHAAsK_CuFnSClCd4Mth3jKzMI4wdLHca4CnllcRS9EbLU_xCawtcMv2Lprd5q9_n271et1e4N-v9Vp4K1Z7jQ7d_1uuz_otbrdXndwaOAfsdNW877fPfwEShWQ4A



Используемые технологии

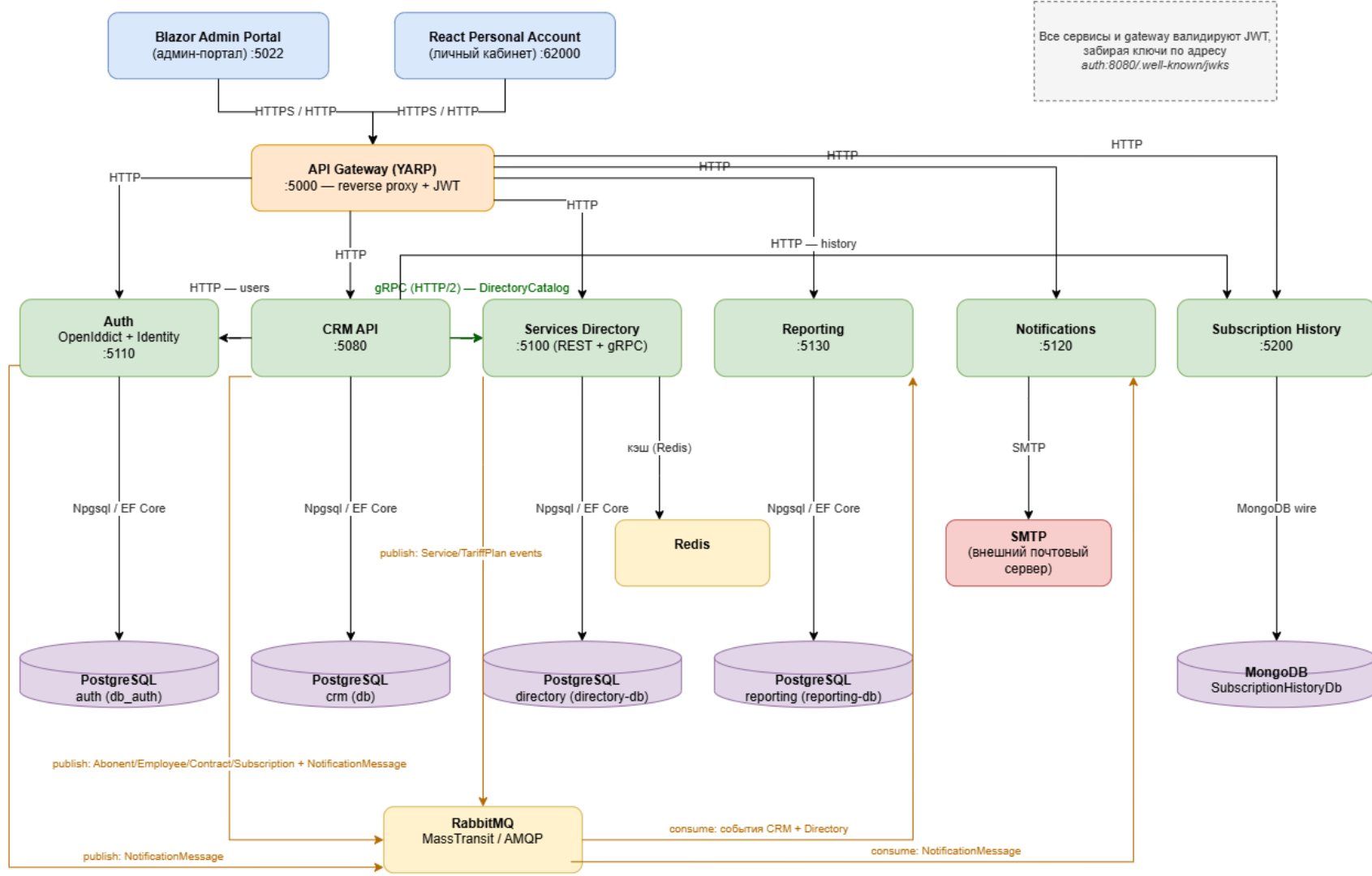
1. Микросервисная архитектура
2. PostgreSQL, MongoDB, EF Core
3. Брокер сообщений: RabbitMQ
4. Фронтенд: Blazor, React.js
5. Docker, GitHub Actions, Scalar, PostgreSQL Adminer, Mongo Express



Продвинутые практики

1. Коммуникация: REST API, gRPC
2. Нестандартный DI через Autofac
3. Валидация: FluentValidation
4. Кастомный Middleware обработки
5. Структурные логи и двухуровневый кэш – IMemoryCache и Redis





PostgreSQL, MongoDB, EF Core

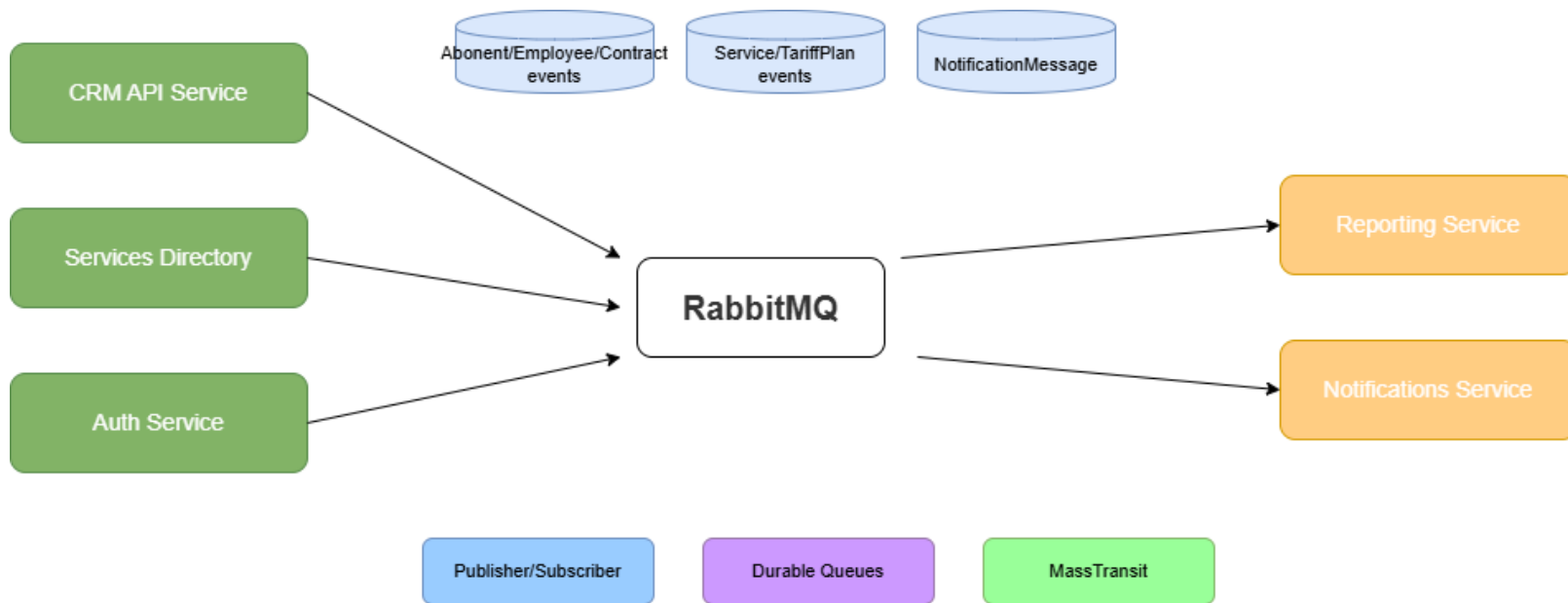
Базы данных и ORM.

- Реляционные данные - PostgreSQL.
- Документные данные - MongoDB.
- Единая ORM – Entity Framework Core.
- Database-per-service паттерн.



Брокер сообщений RabbitMQ

Асинхронная коммуникация.



Брокер сообщений RabbitMQ

Асинхронная коммуникация.

- Брокер RabbitMQ в Docker.
- Паттерн: Publisher/Subscriber.
- События: создание клиента, отчёт.
- Гарантия доставки: durable queues.
- Отвязка сервисов друг от друга.



Фронтенд: Blazor, React.js

Два фронтенд-приложения.

Параметр	Admin Portal	Personal Account
Технология	Blazor WebAssembly	React.js + Vite
Аудитория	Менеджеры компании	Абоненты (клиенты)
Функции	CRM, отчёты, справочники	История подписок, профиль



DevOps-инструменты

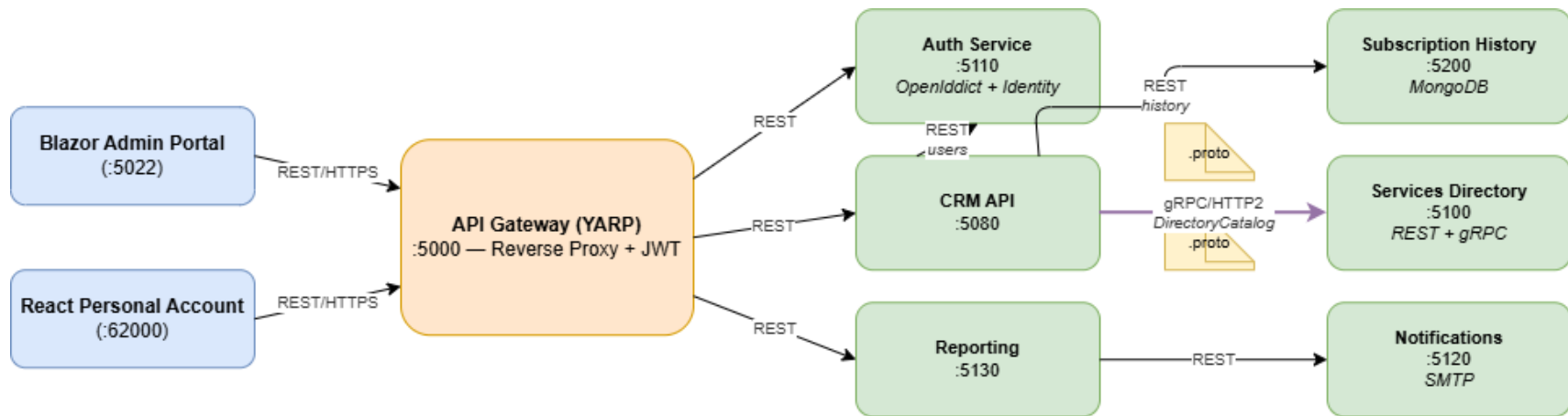
Инфраструктура и CI/CD.

- Docker – контейнеризация всех сервисов.
- GitHub Actions.
- Scalar – современная документация API.
- Adminer – веб-интерфейс для PostgreSQL.
- Mongo Express – UI для MongoDB.



Коммуникация: REST API, gRPC

Протоколы взаимодействия.



Коммуникация: REST API, gRPC

Протоколы взаимодействия.

- REST – для клиентов через Gateway.
- gRPC – между внутренними сервисами.
- Protobuf для компактных сообщений.
- Сильная типизация контрактов.



Нестандартный DI через Autofac

Autofac + Decorator Pattern.

- Замена стандартного контейнера.
- Паттерн Декоратор для ReportService.
- Упрощена регистрация MassTransit.
- Гибкость без изменения кода.



Валидация: FluentValidation

Валидация.

- Отделена от бизнес-логики.
- Легко покрывается тестами.
- Читаемы правила валидации.
- Интеграция через MediatR Pipeline.



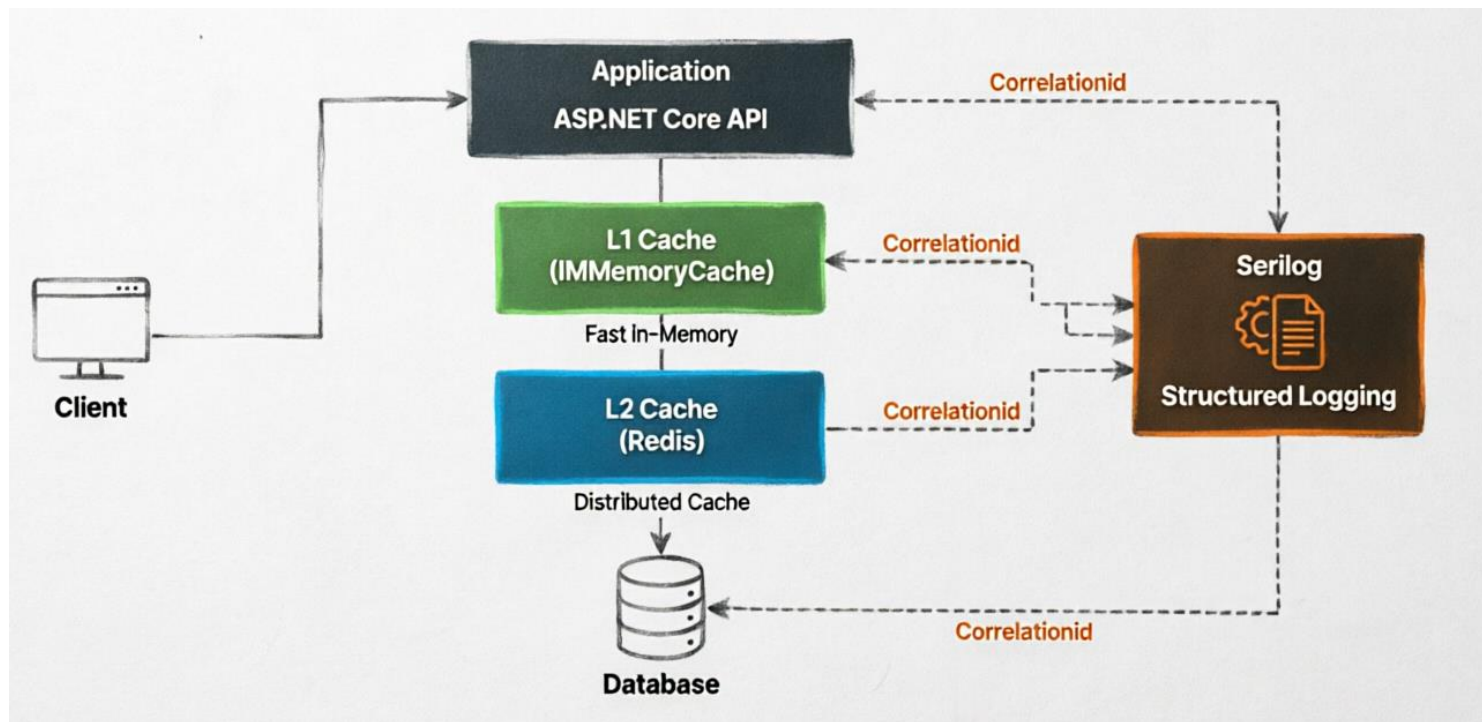
Кастомный Middleware обработки

- Глобальная обработка исключений и ошибок.
- Добавление CorrelationId к каждому запросу.
- Логирование входящих запросов.
- Стандартизация ответов API.



Структурные логи и двухуровневый кэш

IMemoryCache и Redis.



Структурные логи и двухуровневый кэш

IMemoryCache и Redis.

- L1: IMemoryCache – быстрый in-progress.
- L2: Redis – распределенный между сервисами.
- Serilog с обогащением контекста.
- CorrelationId для трассировки.



Демонстрация проекта

Выводы

- **Цель достигнута**

Создана рабочая микросервисная система из 6 сервисов, настроен CI/CD через GitHub Actions.

- **Ключевые результаты**

Главное достижение — разнообразие коммуникаций: REST для внешних клиентов, gRPC для внутренних вызовов, RabbitMQ для асинхронных событий. Плюс двухуровневый кэш и структурное логирование с CorrelationId для наблюдаемости



Ответьте на вопросы
одногогруппников и
преподавателей и получите
обратную связь на свою работу

Вопросы и рекомендации



если есть вопросы



если вопросов нет



Спасибо за внимание!

OTUS

